RethinkVC

Project Team

Mubeen Qaiser	21I-0788
Muneeb Ur Rehman	21I-0392
Muhammad Sohail Shahbaz	21I-1356

Session 2021-2025

Supervised by

Dr. Muhammad Arshad Islam

Department of Computer Science

National University of Computer and Emerging Sciences
Islamabad, Pakistan

June, 2022

Contents

List of Figures

List of Tables

Chapter 1

Introduction

Version Control Systems tracks and manages changes made to text files. And among version control systems, Git is by far the most used version control system [3]. The key features considered when creating Git was; simple design and non-linear development through branching [1]. Git was created in 2005 [1], a time when collaboration meant asynchronous communication and frequent reviews. Today, almost everyone in the world is online and working in real-time, so it is time to rethink version controlling [2].

With the introduction of our project, we propose a real-time environment with tracked changes to bridge the gap between Git version control and real-time editing.

1.1 Problem Statement

Git is a popular version control system with key features simple design and non-linear development through branching. However, Git was created in 2005, a time when collaboration meant asynchronous communication and frequent reviews. Therefore, it comes with a set of limitations according to the today's online and real-time environments.

Frequent merge conflicts, where multiple branches are created to provide a non-linear development environment but results in merging each branch and resolving the conflicts each time.

Delayed feedback, where multiple people are working in different branches and the team leads are unaware of the state at which their project currently is.

Repeated efforts, due to delayed feedback, even the co-works are unaware of the state at which the project is therefore creating a possibility where multiple people might work on the same task achieving the same goal, regardless of being a planned project.

Cumbersome review process, reviewing someone's work requires multiple commands like switch, stash, stash pop, commit, commit –amend, push, and pull, making the process

cumbersome and time-consuming.

Non-intuitive for non-technical users and beginners, where the non-technical users or beginners have to go through a steep learning curve to start using Git as a collaborative tool [4].

1.2 Scope

Multiple users will be able to edit files concurrently and seamlessly. Users' real-time edits will be tracked and contain a visual indicating who made the change. The tracked changes will be used to commit to Git in the backend through an Graphical User Interface. The tracked changes will be removed locally and/or in database once they are added to a commit. Git's powerful commands such as branch, checkout, diff, revert and many more will be available to the user through our GUI. The creator of the project can assign different multiple collaborators with different access. There are 3 major roles; viewer, commenter and editor. The creator of the project or collaborators with extra access can review and approve their teammates work through tracked changes. The approval will lead to a commit in Git. The project will be available as a web application and a Visual Studio Code Plugin. The web application will be mainly for non-technical users with real-time collaboration and git integration. The Visual Studio Code Plugin will be for technical users with real-time collaboration and git integration.

1.3 Modules

1.3.1 Real-Time Collaboration

1. Collaborative text editing with real-time sync between users working on the same file
2. Show live cursors and selections to indicate where other users are editing
3. Follow mode to track other users' movements
4. Individual undo/redo stack for each user
5. Enable/Disable content lock to prevent overlapping edits

1.3.2 Role-Based Access and Session Sharing

1. Manage access control through role-based permissions and timed sessions

2. The host can assign three types of roles to the users:
   (a) Viewer
   (b) Commenter
   (c) Editor
3. The host can apply a time limit on user access after which their session will be finished
4. The host can invite people by email or through a shareable link

1.3.3 Git Integration

1. Store Git in the backend
2. Provide a visual interface to manage branches, commits
3. Users can compare commits and branches
4. Users can revert to previous versions of the document
5. Users can check out different branches and/or commits
6. Users can stash changes temporarily

1.3.4 Tracking Real-time Changes

1. Track real-time changes efficiently
2. Visualize the changes made by different users
3. Users can discard changes made by other users if authorized
4. Users can prepare a commit to Git

1.3.5 Code/Text Management

1. The host can accept/reject the user changes that are committed by the user and only accepted commits integrated into the document and version control (Git)
2. Users can create multiple files within a folder for better document management in our web app

1.4 User Classes and Characteristics

Example: User Classes and Characteristics

User Class	Description
Administrator	The Administrator has full control over the platform, including managing roles, permissions, system-wide settings, and overseeing all projects. They can create, delete, and modify projects or sessions, review commits, approve changes, and configure Git integration. Administrators are responsible for ensuring smooth operation and performance of the system.
Project Manager	The Project Manager oversees specific projects, managing team members, assigning roles (e.g., Editor, Commenter, Viewer), and reviewing project progress. They are responsible for tracking changes, managing real-time collaboration sessions, and handling Git-related workflows like branching, merging, and approving commits.
Editor	Editors are responsible for making changes to project content in real-time, with tracked edits. They can commit changes to version control (Git), manage branches, compare commits, and revert to previous versions. Editors have the ability to make and review changes directly in the content.
Commenter	Commenters provide feedback on project content but do not have editing capabilities. They can view content, suggest improvements, and participate in real-time collaboration by tracking changes and providing input, without committing or approving changes.
Viewer	Viewers have read-only access to the content and Git history. They can view the real-time changes made by other users but cannot make edits, leave comments, or interact with the content. Viewers are typically stakeholders or external parties who need to monitor progress.
Guest User	Guest Users are temporary collaborators with limited access, typically invited to specific sessions. Depending on their role (Viewer, Commenter), they may have restricted access for a set period. Guest Users are typically external stakeholders or partners invited to participate in short-term projects.

Chapter 2

Project Requirements

This chapter describes the functional and non-functional requirements of the project.

2.1 Use Case Diagram

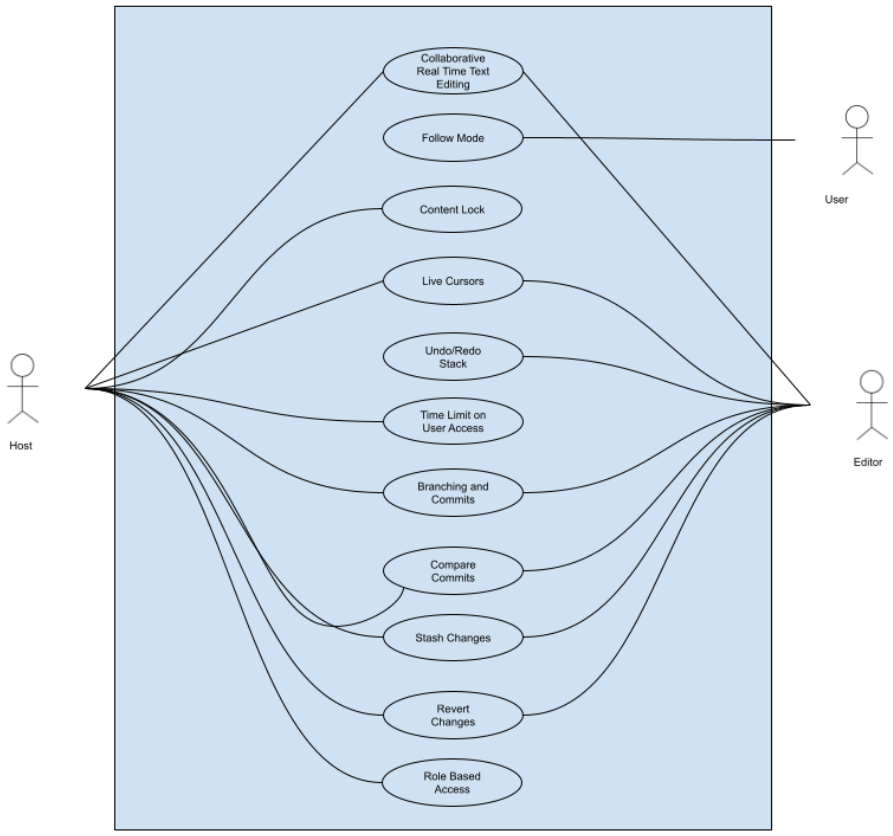

Figure 2.1: Use Case Diagram

2.2 Detailed Use Cases

2.2.1 Collaborative Text Editing

Actors: Host, Editor
Description: Multiple users collaborate on editing the same document in real-time.

Preconditions:

- Users must be authenticated.
- A real-time session must be active.
- The user must have editing permissions (Editor or Host).

Main Flow:

1. The user opens a document in the system.
2. The system syncs the user's view with the active document state.
3. The user makes edits, which are instantly reflected in all other collaborators' views.
4. Changes made by each user are tagged with their respective username.
5. All edits are stored in the system's temporary memory for future commits.

Postconditions:

- The document is continuously updated with all users' changes.
- Edits remain temporary until they are committed to Git.

2.2.2 Show Live Cursors and Selections

Actors: Host, Editor, Viewer, Commenter
Description: The system displays other collaborators' cursors and text selections in real-time.

Preconditions:

- The user must be part of an active session.
- The user must have a view of the document.

Main Flow:

1. The user starts editing the document.
2. The system tracks the cursor position and text selections for each user.
3. The system displays these cursors and selections in real-time to all collaborators, marked by the user's name or color.

Postconditions:

- Each user sees the live cursor movements and text selections of other collaborators.

2.2.3 Follow User Movements

Actors: Host, Editor, Commenter, Viewer
Description: A user can follow another user's movements within the document to track their edits in real-time.

Preconditions:

- The session must be active.
- The user must be authenticated and have view access.

Main Flow:

1. The user selects another user's name from the list of active collaborators.
2. The system locks the user's view to the selected collaborator's cursor movements.
3. The user can observe the selected collaborator's real-time actions and edits.
4. The user can stop following at any time.

Postconditions:

- The user's view follows the selected user's actions in real-time.

2.2.4 Individual Undo/Redo Stack

Actors: Editor
Description: Each user can undo or redo only their own changes in the document.

Preconditions:

- The user must be an Editor or Host.
- The session must be active.

Main Flow:

1. The user makes edits in the document.
2. The user performs an undo action (Ctrl + Z).
3. The system reverts only the user's own changes.
4. The user can perform a redo action (Ctrl + Y) to bring back their undone changes.

Postconditions:

- Only the user's edits are undone or redone. Other users' edits are unaffected.

2.2.5 Enable/Disable Content Lock

Actors: Host, Editor
Description: Prevents overlapping edits by enabling a content lock, ensuring only one user can edit a portion of the document at a time.

Preconditions:

- The user must have Editor or Host permissions.
- The session must be active.

Main Flow:

1. The user enables the content lock.
2. The system prevents other users from editing the currently locked section.
3. The user can disable the lock after completing their edits.

Postconditions:

- The locked content remains uneditable by others until the lock is released.

2.2.6 Manage Role-Based Access

Actors: Host
Description: The host assigns roles (Viewer, Commenter, Editor) to other users and controls their level of access.

Preconditions:

- The host must be authenticated and have access to manage collaborators.

Main Flow:

1. The host opens the access management panel.
2. The system lists all collaborators.
3. The host assigns roles to the selected users (Viewer, Commenter, Editor).
4. The system applies the assigned roles and updates permissions accordingly.

Postconditions:

- Users can access or modify the document based on their roles.

2.2.7 Apply Time Limits on User Access

Actors: Host
Description: The host can set a time limit on a collaborator's access to the project.

Preconditions:

- The session must be active, and the host must be authenticated.

Main Flow:

1. The host selects a collaborator and sets a time limit for access.
2. The system applies the time limit to the selected user.
3. Once the time expires, the system terminates the user's session.

Postconditions:

- The user's session expires after the specified time, and they lose access to the document.

2.2.8 Invite Users via Email/Shareable Link

Actors: Host
Description: The host invites users by email or shareable link to collaborate on the project.

Preconditions:

- The host must be authenticated and have access to the project's invite functionality.

Main Flow:

1. The host selects the option to invite users.
2. The system generates an invitation link or sends an email to the user.
3. The invited user accepts the invitation and joins the collaboration session with the assigned role.

Postconditions:

- The invited user joins the project with assigned access rights.

2.2.9 Store Git in Backend

Actors: Git System
Description: The system stores project files and their version history in a Git repository on the backend.

Preconditions:

- The project must be integrated with Git.

Main Flow:

1. The user edits or commits changes to the document.
2. The system stores the new version in the Git repository.
3. The system updates the commit history.

Postconditions:

- The document is stored in Git, and its version history is available for review.

2.2.10 Visual Interface for Branches and Commits

Actors: Host, Editor
Description: Provides a visual interface for managing branches and commits.

Preconditions:

- The project must be integrated with Git.

Main Flow:

1. The user opens the Git interface.
2. The system displays the available branches and commits.
3. The user can switch between branches, create new ones, or view commit histories.

Postconditions:

- The user can visually manage branches and commits.

2.2.11 Compare Commits and Branches

Actors: Host, Editor
Description: The user can compare differences between commits or branches.

Preconditions:

- The project must be integrated with Git.

Main Flow:

1. The user selects two commits or branches.
2. The system compares the selected commits or branches and displays the differences.

Postconditions:

- The differences between the selected commits or branches are displayed.

2.3 Event Response Table

Event	State	Response
User renames project	Session is on	Rename the project
User deletes project	Session is on	Ask user to first close any session related to the project
User deletes project	Session is off	Ask for confirmation, when confirmed delete the project
User starts session	No sessions are on	Notify all the collaborators of the project of session start
User starts session	Session already exists	• Inform user session already exists • Give option to be redirected to session
User extends session time	Any	• Extend session time • Notify all collaborators the time extension
User closes session	• Files are edited • Edited files are not committed to git	Save all real-time edits and content locks to database
Session time interval has ended	• Files are edited • Edited files are not committed to git	Save all real-time edits and content locks to database
User creates file	Project is idle	Create file without adding to git
User renames file	• Session is on • Collaborators are working on the file	Rename the file

Continued on next page

Event	State	Response
User deletes file	• Session is on • Collaborators are working on the file	• Archive all real-time edits • Archive all content locks • Archive all comments • Delete the file
User moves file	• Session is on • Collaborators are working on the file	Move the file
User edits file	• Session is on • User has editorial rights • Does not overlap other user's content	Accept edits
User edits file	• Session is on • User has editorial rights • Overlaps other user's content • No content lock on user's content	Accept edits
User enables lock content	• Session is on • User has editorial rights • User has made edits	Selected content will be locked
User edits file	• Session is on • User has editorial rights • Overlaps other user's content • Content lock on user's content	• Edit will not be allowed • Display a message that content lock is enabled

Continued on next page

Event	State	Response
User adds changes to working directory	• Session is on • User has editorial rights • One user's edits are selected	Add changes to git's working directory
User adds changes to working directory	• Session is on • User has editorial rights • One user's edits are already in working directory • Another user's edits are selected	• Queue the edit to temporary working directory • Display a message of the queue
User commits changes	• Session is on • User has editorial rights • Dequeue from the queue of working directories	• Commit changes • Remove real-time tracked changes associated with the commit

Table 2.1: Event Response Table

2.4 Functional Requirements

This section lists the functional requirements organized by the modules. Each functional requirement is written in natural language.

2.4.1 Module 1: Real-Time Collaboration

FR-1: The system shall allow the user to collaborate with other users in a single file in real-time.

FR-2: The system shall display the user's cursor in real-time during collaboration.

FR-3: The system shall display the user's text selection in real-time during collaboration.

FR-4: The system shall allow the user to view the list of active collaborators in the project.

FR-5: The system shall allow the user to follow selected user's edits in real-time during collaboration.

FR-6: The system shall allow the user to undo/redo only their own changes during collaboration.

FR-7: The system shall allow the project creator, admin and editor to be able to enable a lock feature where their real-time changes can not be edited by another user.

FR-8: The system shall allow the project creator, admin, editor and commenter to comment on the project files.

FR-9: The system shall track the changes made by all of the editors in a session and sync it with all the participants.

2.4.2 Module 2: Role-Based Access and Permissions

FR-1: The system shall allow the user to create, rename, delete projects.

FR-2: The system shall allow the project creator and admin to manage roles (Admin, Editor, Commenter, Viewer) to collaborators.

FR-3: The system shall allow the project creator to invite others users via email and assign them roles (Admin, Editor, Commenter, Viewer).

FR-4: The system shall allow the project creator and admin to invite other users (guests) by generating and sharing a link and assign them roles (Commenter, Viewer) for a certain time ranging from 0 seconds to 24 hours specified by project creator.

FR-5: The system shall allow the project creator and admin to cancel or update the time specified for guest user.

FR-6: The system shall allow the project creator, admin and editor to add, rename, move, delete and organize files into folders within a project.

FR-7: The system shall provide the user a markdown interface for editing text.

FR-8: The system shall allow the project creator or admin to safely demote a user role.

2.5 Non-Functional Requirements

This section specifies non-functional requirements. These quality requirements are specific, quantitative, and verifiable to ensure optimal system performance and user satisfaction.

2.5.1 Reliability

REL-1: The real-time tracked changes shall be saved every 30 seconds.

2.5.2 Usability

Usability ensures the system is easy to learn and efficient to use, minimizing user errors and optimizing the user experience during real-time collaboration.

USE-1: Guided popups shall be displayed to users within 1 second upon interaction with new features or complex tasks, providing contextual assistance with up to 3 steps per task to ensure clarity without overwhelming the user.

USE-2: Generic tasks such as adding comments, viewing comments, or managing roles shall be completed within a maximum of 3 clicks, ensuring task efficiency.

USE-3: Active users shall be displayed in real-time, updating every 5 seconds to show the current number of users interacting with the system.

USE-4: Notifications shall be provided to users within 2 seconds of any relevant system actions or updates, with no more than 3 notifications shown at once to avoid overload.

USE-5: Tooltips shall be displayed within 0.5 seconds when users hover over icons or buttons, providing brief descriptions to enhance user understanding.

2.5.3 Performance

Performance ensures that the system is responsive and meets time-based requirements for real-time collaboration tasks and version control operations.

PER-1: The system shall queue simultaneous commits and display the commit queue for 10 seconds after the last commit attempt.

PER-2: The system shall issue a warning for potential performance degradation when a file exceeds 90 percent of the 100,000-character limit.

PER-3: The system shall issue a warning for potential performance degradation when a file exceeds 1,000 lines.

PER-4: The system shall issue a warning for potential performance degradation when a file exceeds 5 MB in size.

PER-5: The system shall issue a warning for potential performance degradation when more than 5 users are editing the same file simultaneously.

2.5.4 Security

Security ensures the protection of the system and its data by enforcing robust access control, integrating securely with Git for version management, and providing strong user authentication mechanisms to prevent unauthorized access.

SEC-1: Changes in roles shall be reported to the creator immediately through email and notification.

SEC-2: Deletion or removing all the content of more than 5 files within 24 hours shall be reported to both the admin and the creator immediately through their emails and notification.

SEC-3: Deletion or merging of branches, or committing of changes to Git shall be reported to the creator through their email and notification.

SEC-4: Edits and commits shall be protected by role-based access control at all times.

2.6 Domain Model

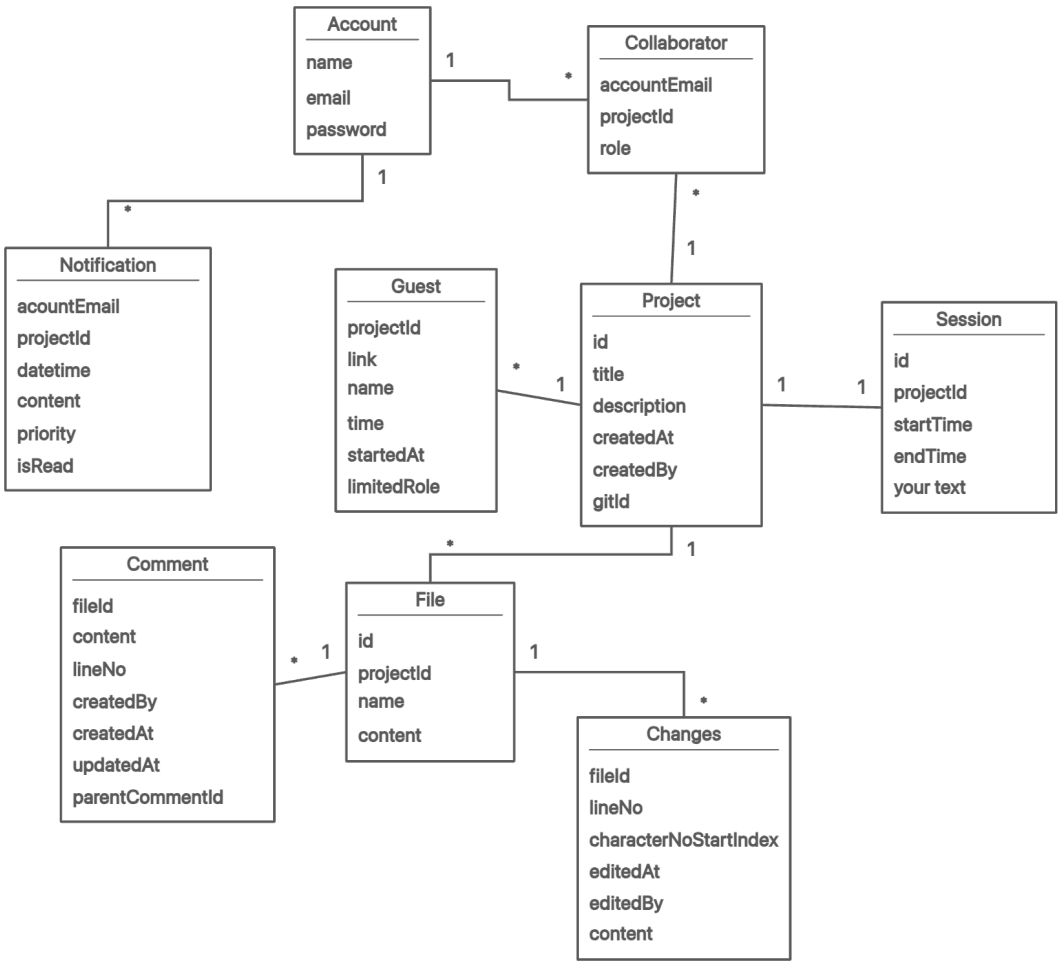

Figure 2.2: Domain Model

Chapter 3

System Overview

3.1 Architectural Design

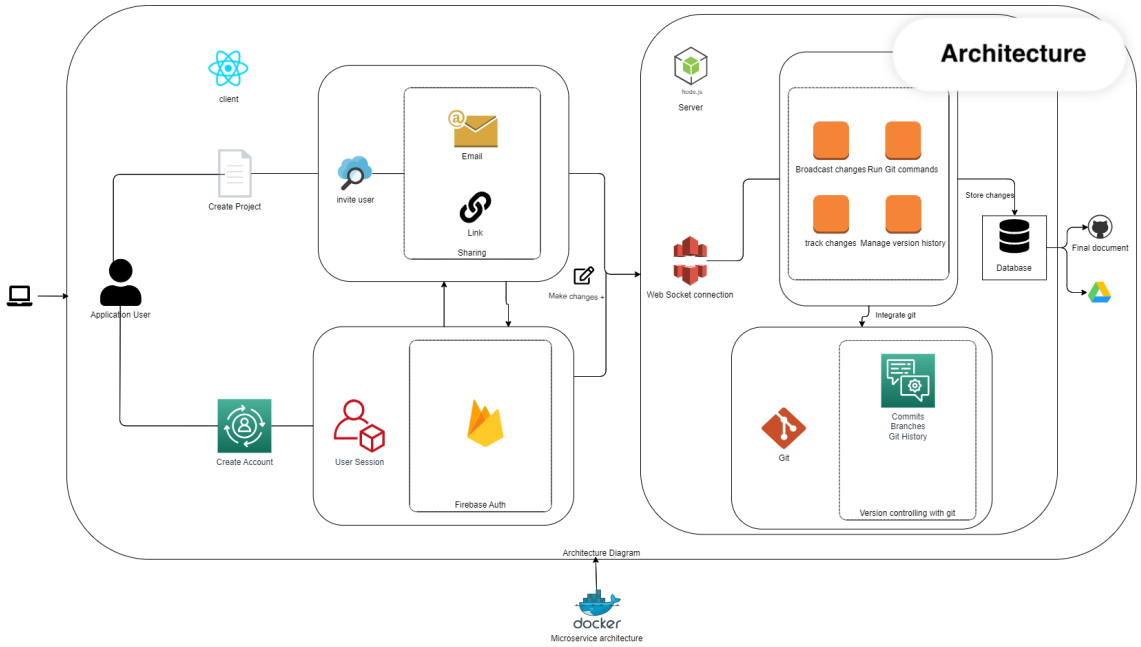

Figure 3.1: Architecture Diagram

3.2 Design Models

3.2.1 Activity Diagram

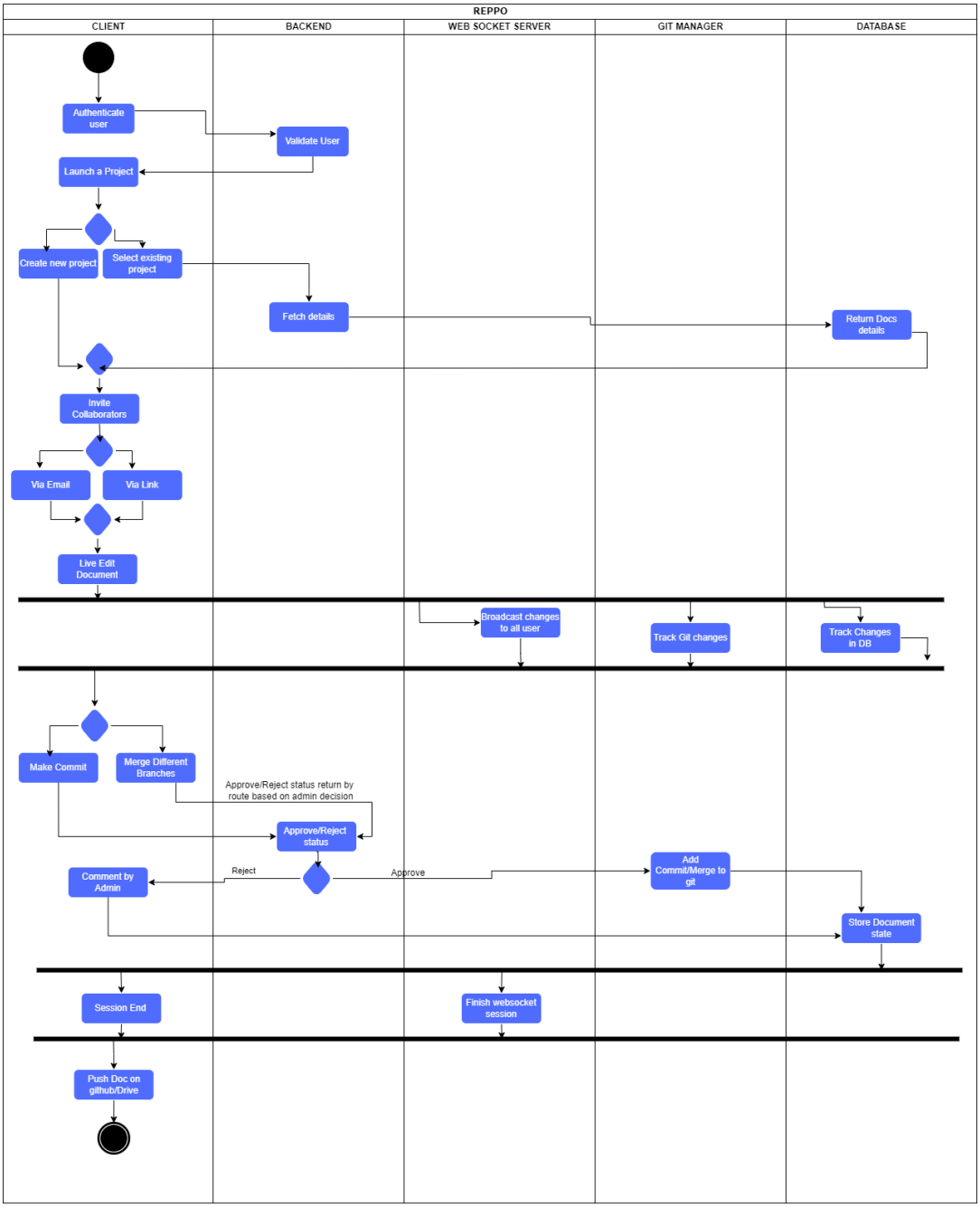

Figure 3.2: Activity Diagram

3.2.2 Class Diagram

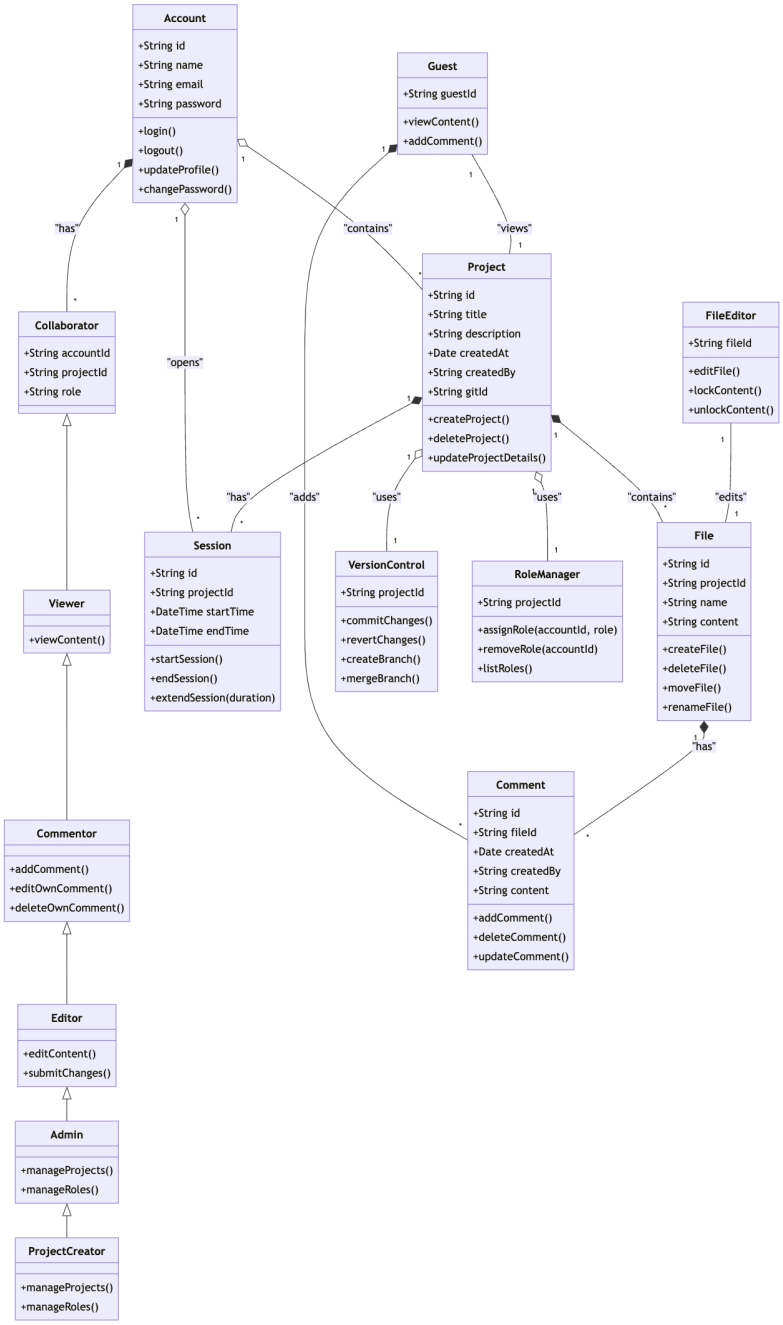

Figure 3.3: Class Diagram

3.2.3 Class-level Sequence Diagrams

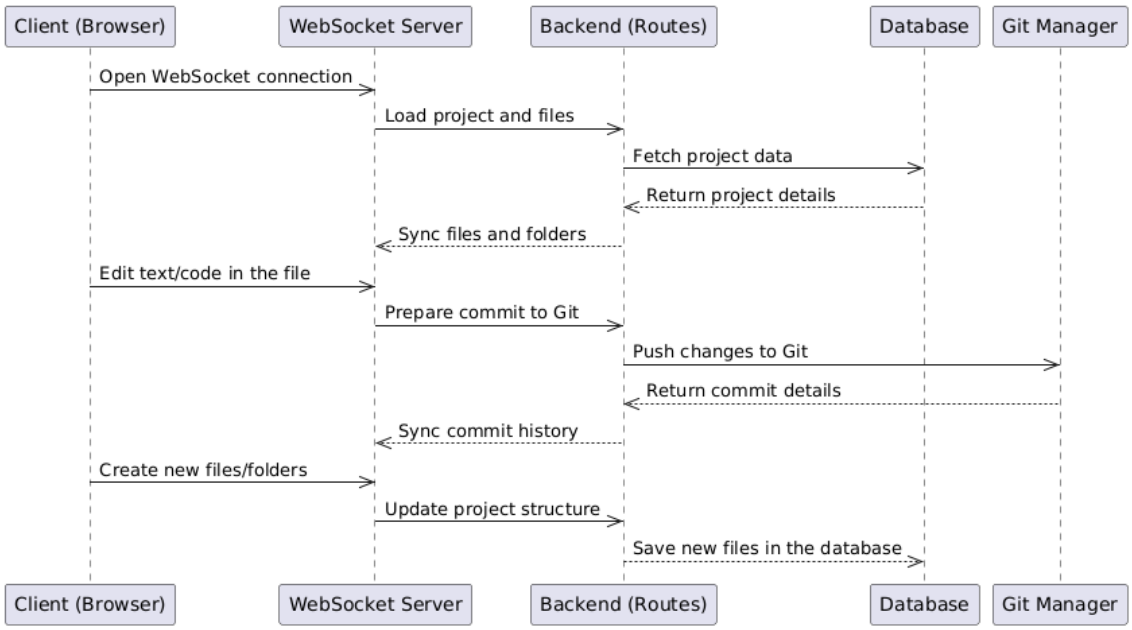

Figure 3.4: Code:Text Management

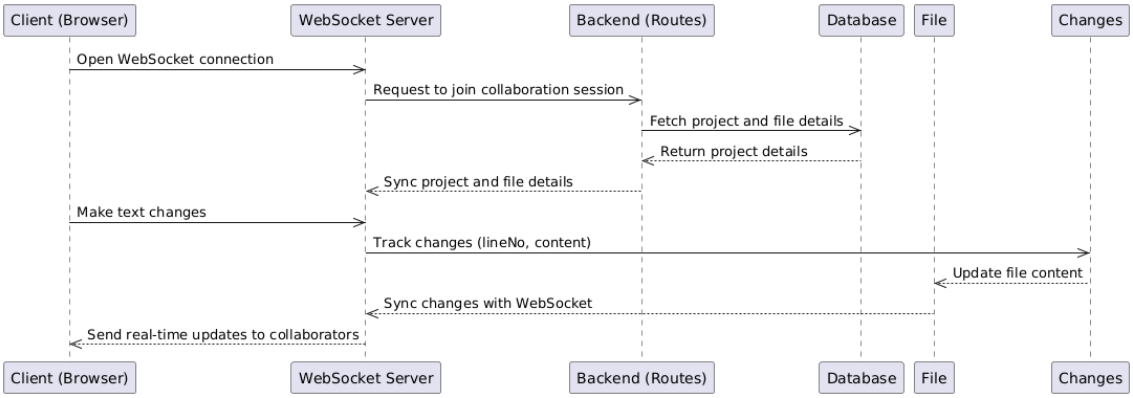

Figure 3.5: Collaborative Text Editing with Real-Time Sync

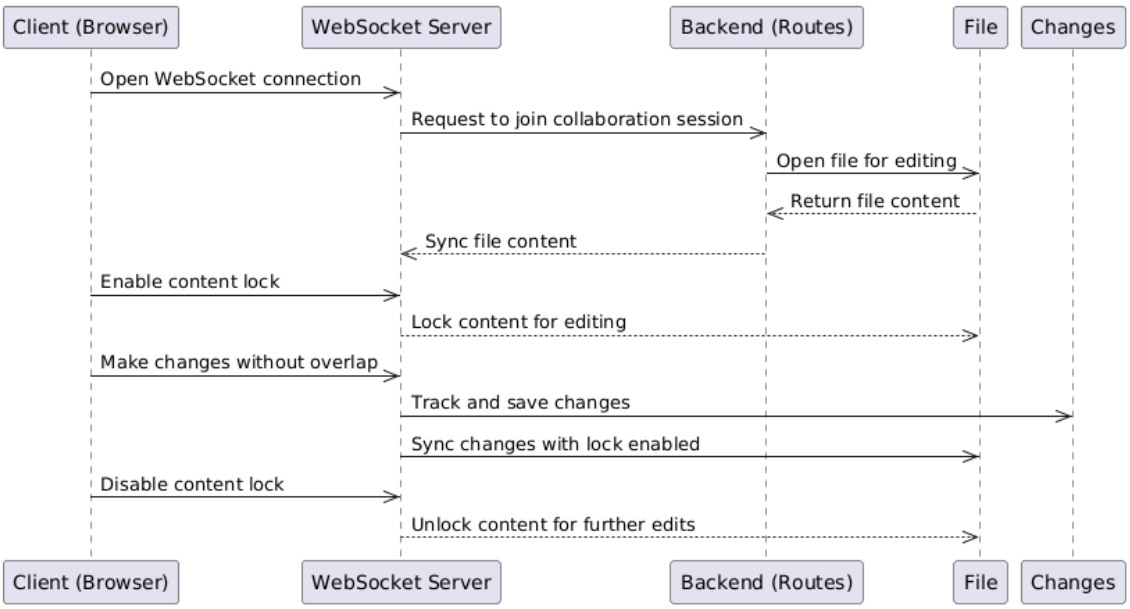

Figure 3.6: Enable:Disable Content Lock to Prevent Overlapping Edits

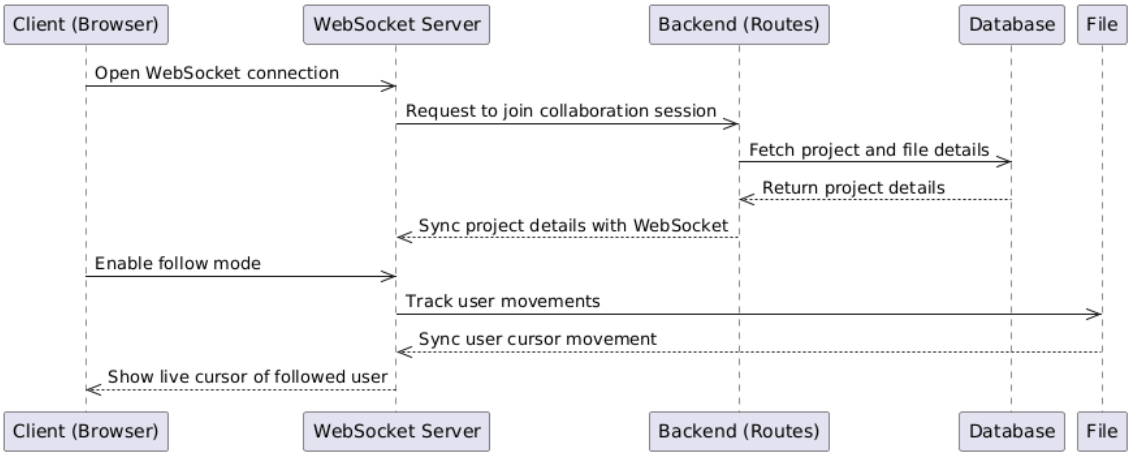

Figure 3.7: Follow Mode to Track User Movements

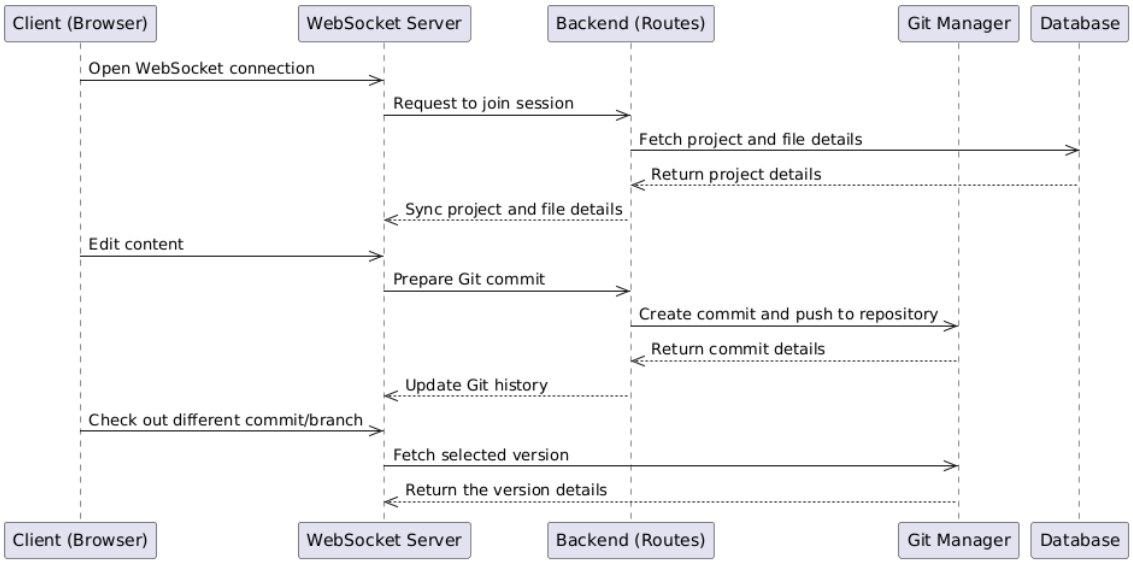

Figure 3.8: Git Integration

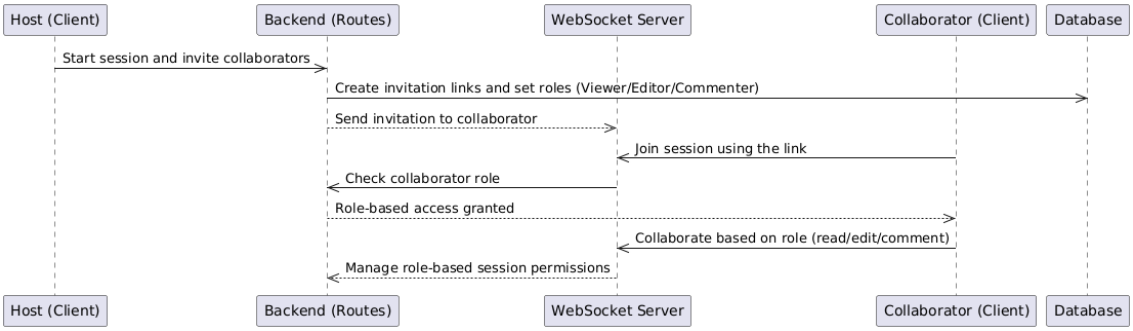

Figure 3.9: Role-Based Access and Session Sharing

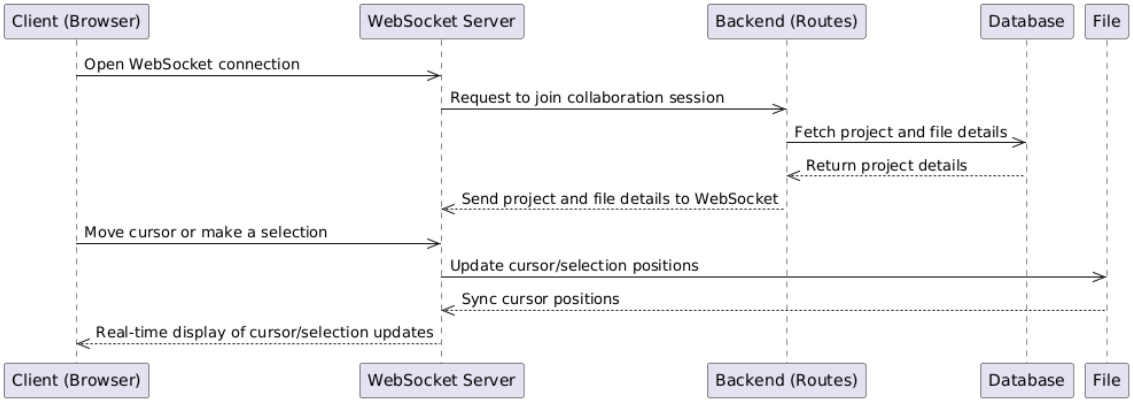

Figure 3.10: Show Live Cursors and Selections

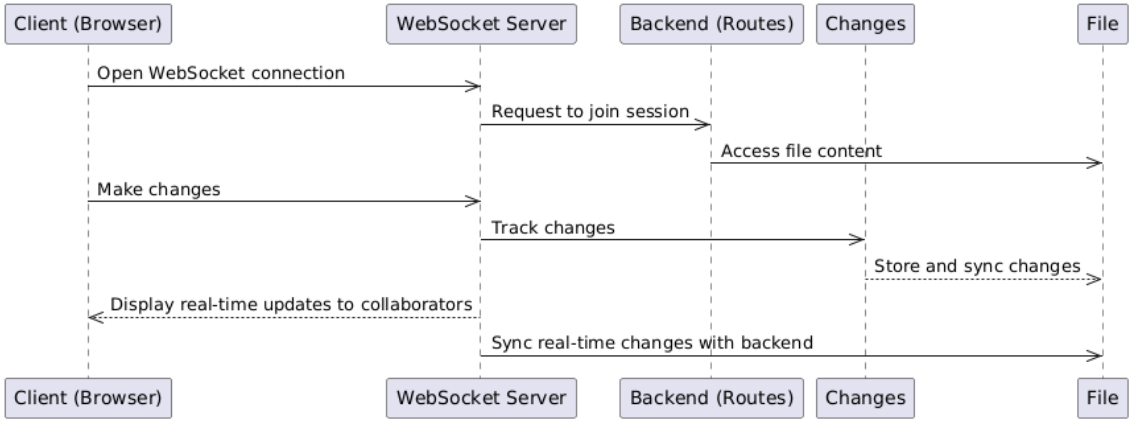

Figure 3.11: Tracking Real-Time Changes

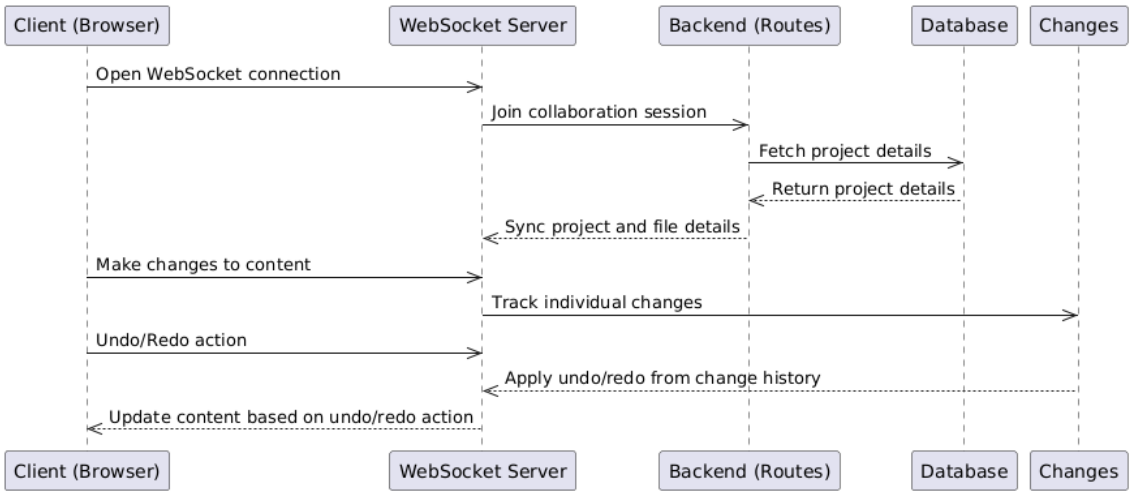

Figure 3.12: Undo and Redo Stack

3.2.4 State Transition Diagram

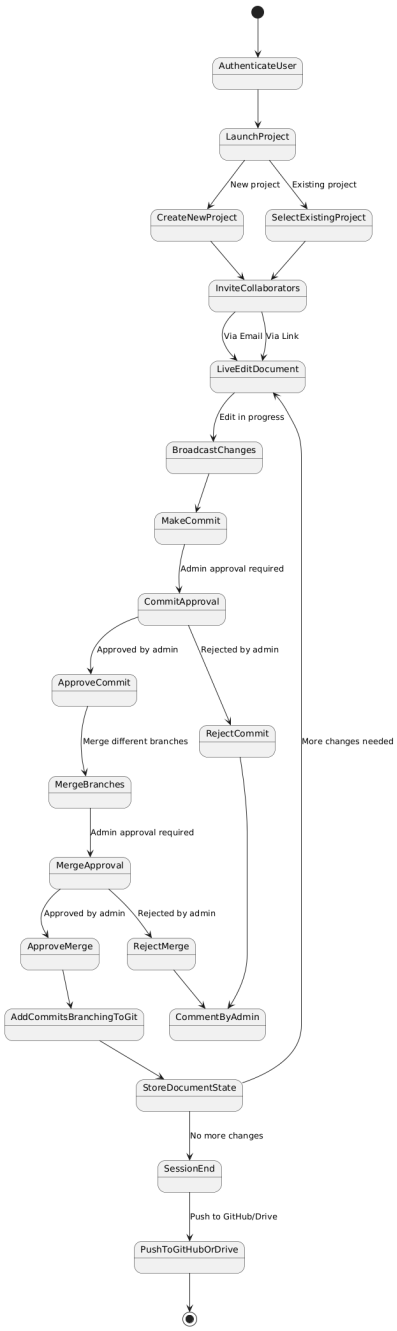

Figure 3.13: State Transition Diagram

3.3 Data Design

In this system, data flows through different components with a focus on **real-time collaboration**, **state management**, and **version control**. The design ensures efficient handling of real-time data, edits state synchronization across users, and periodic storage of persistent data.

3.3.1 Data Flow and Processing

1. **File Loading**: When a session starts, files are loaded from the database (**Supabase**) and delivered to all users involved in the session as the login and access the project. The next upcoming udpates are fetched from cached storage using **Redis**.

2. **Real-Time Collaboration**:

   - **Socket.IO** is used to manage real-time communication between users. As users make edits, these changes are transmitted through WebSockets to other users in the session.
   - **Redis** acts as a temporary in-memory cache for the real-time state, reducing the need for frequent database writes. It ensures that real-time changes are quickly accessible and synchronized across all users.
   - Each user's local storage also maintains a copy of the current session state, ensuring continuity even during network disconnections.

3. **State Synchronization**:

   - Real-time edits from users are stored locally and synced across all clients using **Socket.IO**.
   - Every 30 seconds, the current real-time state is flushed from **Redis** and local storage and persisted into **Supabase**, ensuring data integrity.

4. **Commit and Version Control**:

   - Once the session ends or a commit is triggered, all unsaved edits are committed to the **Git repository** via the **Git Integration Service**.
   - A queue system can be used here to queue edits/commits, ensuring orderly processing of changes and reducing conflicts during large collaboration sessions.

3.3.2 Data Storage Items

- **Supabase** (PostgreSQL): For storing persistent project data, such as projects, session metadata, and file versions. Data is updated periodically (every 30 seconds) to ensure synchronization.
- **Redis**: For caching real-time file states, providing fast access to the most current version of the file being edited. Redis ensures minimal delay between user edits.
- **Local Storage**: Used in the browser to maintain the real-time session state for each user, ensuring smooth user experience even in cases of brief disconnections.
- **Git**: Manages version control, including committing changes, branching, and merging. Commits are queued and processed once finalized to ensure changes are properly reflected in the version history.

3.3.3 Data Handling and Queues

Message Queues: Used for managing edits, ensuring changes are processed in order. This helps in handling high-frequency events (real-time edits) while avoiding conflicts. Queues ensure that commits are processed sequentially without overwhelming the system.

3.3.4 Entity Relation Diagram

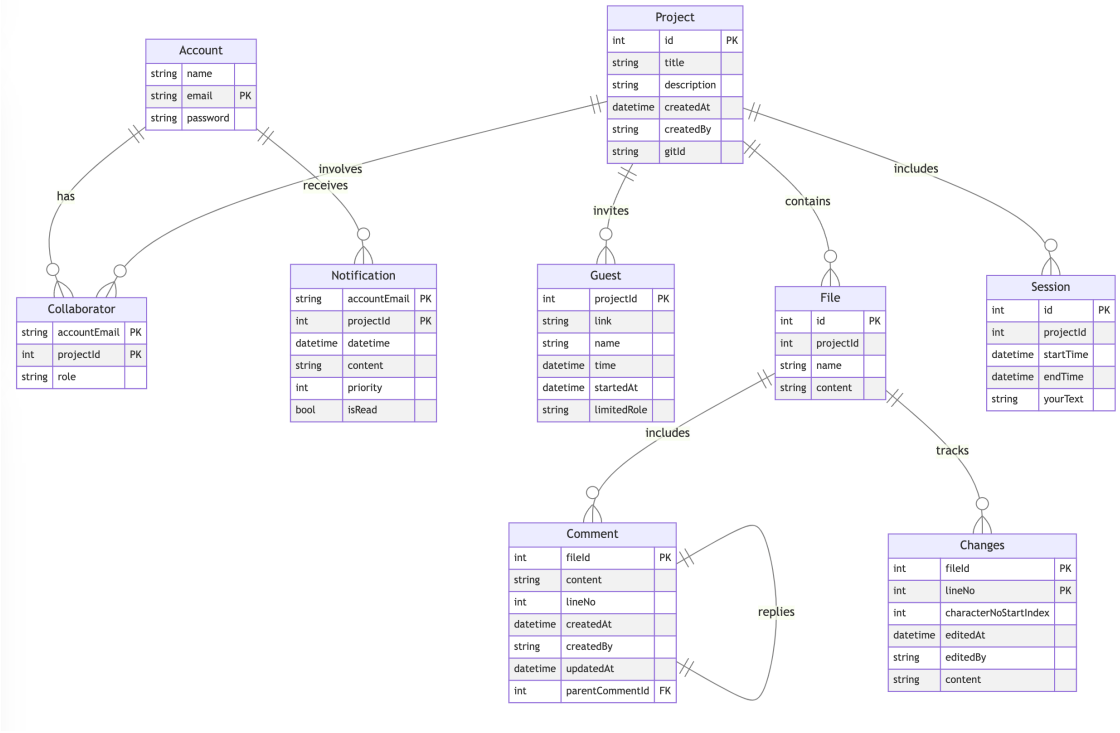

Figure 3.14: Entity Relation Diagram

Bibliography

[1] Scott Chacon and Ben Straub. Progit. *https://git-scm.com/book/en/v2/Getting-Started-A-Short-History-of-Git*, 2024. Updated on 2024-09-04.

[2] Yousuf Jawwad. Rethinking version control for real-time collaboration. *https://fatalexception.substack.com/p/rethinking-version-control-for-real?r=alqnctriedRedirect=true*, 2024, May. Accessed: 2024-09-12.

[3] Stack Overflow. Stack overflow developer survey 2022. *https://survey.stackoverflow.co/2022/section-version-control-version-control-systems*, 2022. Accessed: 2024-09-12.

[4] Emily von Hoffmann. Learning curve is the biggest challenge developers face with git. *https://about.gitlab.com/blog/2017/05/17/learning-curve-is-the-biggest-challenge-developers-face-with-git/*, 2017, May 17. Accessed: 2024-09-12.